Akshansh Agrawal
SRN: PES2UG24CS045

Lab 1: Requirements Engineering & UML Use Case Modelling

Problem Statement #41: OpenAPI Mock Server Generator

About the System

An OpenAPI Mock Server Generator reads an existing OpenAPI 3.0 spec and turns it into a working fake API almost instantly, so developers and testers don't have to wait on a real backend to start working. It generates realistic, schema-compliant responses for every path in the spec and can *simulate response delay* to mimic real network conditions. Wired into a **CI/CD pipeline**, it also gives test suites something stable to run against instead of a backend that might be incomplete or down.

Actors

Five actors interact with the system in different ways, some directly and some in the background.

Actor	Type	Description
API Developer	Primary	Uploads the OpenAPI spec, sets up latency behavior, and is basically the person configuring the mock in the first place.
QA Engineer	Primary	Sends requests against the mock endpoints while testing and depends on the responses acting like the real API would.
CI/CD Pipeline	Secondary	An automated system that spins the mock server up during test runs so builds don't have to wait on the real backend.
System Administrator	Secondary	Handles access control, environment setup, and general server configuration for the mock server instance.
DevOps Engineer	Secondary	Keeps an eye on uptime and load handling, especially when the mock server is getting hit hard during peak test runs.

Requirements Table

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	Parses an OpenAPI 3.0 spec and creates mock HTTP endpoints returning randomized, schema-compliant JSON.	High	Endpoint returns correctly typed JSON. An undefined path must not return 200 OK.	Core function of the tool.
FR-002	Functional	Validates uploaded specs and rejects broken YAML or JSON with a line-numbered error.	High	Invalid file shows an error with a line number. Valid file loads with zero errors.	Stops bad specs from causing silent failures.
FR-003	Functional	Lets the API developer set a minimum and maximum response delay per endpoint.	Medium	Delay set to 200-300ms, 95% of responses fall in that range.	Needed for realistic timeout testing.
FR-004	Functional	Shows the QA engineer a live list of active endpoints with path and method.	Medium	List matches the spec exactly.	Testers need to know what's available.
FR-005	Functional	Generates randomized responses that still match schema types and required fields.	High	100 calls in a row, all responses pass schema validation.	A mock that breaks the contract is useless.
NFR-001	Nonfunctional	Engine sustains 1,000 requests per second with latency accurate to plus or minus 10ms.	High	Benchmark confirms latency and security targets under peak load.	Has to hold up in CI pipelines.
NFR-002	Nonfunctional	System stays available for health checks at least 99.5% of the time per session.	Medium	Uptime monitor logs 99.5% or higher over an 8-hour run.	A flaky mock server breaks CI runs.

Use Case Flow

Use cases: UC-01 Upload Spec, UC-02 Parse & Validate Spec, UC-03 Generate Mock Endpoints, UC-04 Configure Latency, UC-05 Send Request to Mock Endpoint, UC-06 Validate Response Against Schema.

UC-05 «includes» UC-06, since validation always happens. UC-04 «extends» UC-03, since latency is optional.

Use Case	UC-05: Send Request to Mock Endpoint
Primary Actor	QA Engineer
Secondary Actor(s)	API Developer, CI/CD Pipeline
Precondition(s)	Spec uploaded and parsed (UC-01, UC-02). At least one mock endpoint is active (UC-03).
Postcondition(s)	Success: valid response returned and logged. Failure: error returned and logged.
Main Success Scenario	1. API developer uploads an OpenAPI spec. 2. System parses and validates it (UC-02). 3. System generates mock endpoints (UC-03). 4. QA engineer sends a request to a mock endpoint. 5. System applies configured latency, if set (UC-04).

	6. System builds a randomized response matching the schema. 7. System validates the response against the schema («include» UC-06). 8. System returns the response with the correct status code. 9. Use case ends successfully.
Alternate Flow	6a. Schema validation fails 6a1. System detects the payload doesn't match the schema. 6a2. Failure is logged. 6a3. System returns a 500 error. 6a4. QA engineer sees the failure on the dashboard.

UML Use Case Diagram

Figure 1: UML Use Case Diagram for the OpenAPI Mock Server Generator, showing all actors, use cases, and stereotype relationships.

